

EECS 31L: INTRODUCTION TO DIGITAL DESIGN LAB LECTURE 2

Salma Elmalaki
salma.elmalaki@uci.edu

The Henry Samueli School of Engineering
Electrical Engineering and Computer Science
University of California, Irvine



LECTURE 2: OVERVIEW

- Logistics
- Number representation
- Deeper insight in Verilog structures with additional examples
- Verilog data types
- Testbench
- Verilog operators
- Time permitted: Some examples for concurrency
- Time permitted: Lab 2 description

LECTURE 2: LOGISTICS

- Lab 1 deadline is Jan 25 at 11:59 pm on canvas
- Please follow the submission guidelines
- Work with your TAs to debug and solve any technical problems
- Advise: Go to the discussion labs prepared!

Number and character representation in Verilog

sign	bit_width	'number_base	value
------	-----------	--------------	-------

- Integer numbers:
 - As a decimal number (signed or unsigned)
 - 253, -56
 - as binary, octal, decimal, or hexadecimal form
 - sign
 - Optional
 - Allowed only with decimal numbers
 - Bit_width
 - The (decimal) number in this field corresponds to the width of the number (number of bits)
 - 'number_base
 - Do not forget the ' apostrophe
 - The base of number: **d** (decimal), **b** (binary), **o** (octal), **h** (hexadecimal)
 - Value
 - Corresponds to the value of the number

Number and character representation in Verilog

Format: N'Bvalue

N = number of bits, **B** = base

N'B is optional but recommended (default is decimal)

Number	# Bits	Base	Decimal Equivalent	Stored
3'b101	3	binary	5	101

Number and character representation in Verilog

Format: N'Bvalue

N = number of bits, B = base

N'B is optional but recommended (default is decimal)

Number	# Bits	Base	Decimal Equivalent	Stored
3'b101	3	binary	5	101
'b11	unsized	binary	3	00...0011

Number and character representation in Verilog

Format: N'Bvalue

N = number of bits, B = base

N'B is optional but recommended (default is decimal)

Number	# Bits	Base	Decimal Equivalent	Stored
3'b101	3	binary	5	101
'b11	unsized	binary	3	00...0011
8'b11	8	binary	3	00000011

- If the number size is smaller than the size specified
 - The size is made up by padding 0's to the left

Number and character representation in Verilog

Format: N'Bvalue

N = number of bits, B = base

N'B is optional but recommended (default is decimal)

Number	# Bits	Base	Decimal Equivalent	Stored
3'b101	3	binary	5	101
'b11	unsized	binary	3	00...0011
8'b11	8	binary	3	00000011
8'b1010_1011	8	binary	171	10101011

- The **underscore character** can be used anywhere after the first character.
 - It adds to the readability.
 - It is normally ignored.

•

Number and character representation in Verilog

Format: N'Bvalue

N = number of bits, B = base

N'B is optional but recommended (default is decimal)

Number	# Bits	Base	Decimal Equivalent	Stored
3'b101	3	binary	5	101
'b11	unsized	binary	3	00...0011
8'b11	8	binary	3	00000011
8'b1010_1011	8	binary	171	10101011
3'd6	3	decimal	6	110

Number and character representation in Verilog

Format: N'Bvalue

N = number of bits, B = base

N'B is optional but recommended (default is decimal)

Number	# Bits	Base	Decimal Equivalent	Stored
3'b101	3	binary	5	101
'b11	unsized	binary	3	00...0011
8'b11	8	binary	3	00000011
8'b1010_1011	8	binary	171	10101011
3'd6	3	decimal	6	110
6'o42	6	octal	34	100010

Number and character representation in Verilog

Format: N'Bvalue

N = number of bits, B = base

N'B is optional but recommended (default is decimal)

Number	# Bits	Base	Decimal Equivalent	Stored
3'b101	3	binary	5	101
'b11	unsized	binary	3	00...0011
8'b11	8	binary	3	00000011
8'b1010_1011	8	binary	171	10101011
3'd6	3	decimal	6	110
6'o42	6	octal	34	100010
8'hAB	8	hexadecimal	171	10101011

Number representation in Verilog

- Some notes about the hex representation
 - Verilog is case sensitive. However, the base number characters or the magnitude can be either case
 - A,B,C,D,E,F,O,X,H,Z are the same as
a,b,c,d,e,f,o,x,h,z

Number representation in Verilog

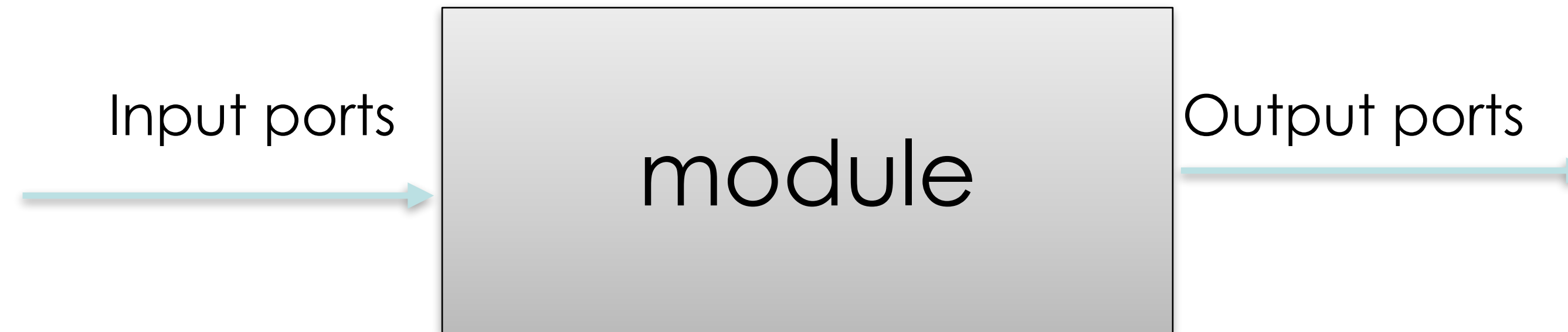
- Real numbers
 - Decimal notation
 - -a.b
 - a and b must be present in decimal number
 - No limit with the number of digits on either side of decimal point
 - Scientific notation
 - There is only one digit on the left side of decimal point
 - The number after e shows the power of 10
 - For example $4.3e2 = 4.3 \times 10^2 = 430$

LECTURE 2: OVERVIEW

- Logistics
- Number representation
- Deeper insight in Verilog structures with additional examples
- Verilog data types
- Testbench
- Verilog operators
- Time permitted: Some examples for concurrency
- Time permitted: Lab 2 description

VERILOG BASICS

- Each design in Verilog consists of one main part:



Structure 1

```

module module_name (port_name1, port_name2, ... );
    port_mode1 port_name1, port_name2;
    port_mode2 port_name3;
//body of module
endmodule
  
```

Structure 2

```

module module_name (port_mode1 port_name1,
port_name2, port_mode2 port_name3);
//body of module
endmodule
  
```

Note the semicolon at the end of the port list!

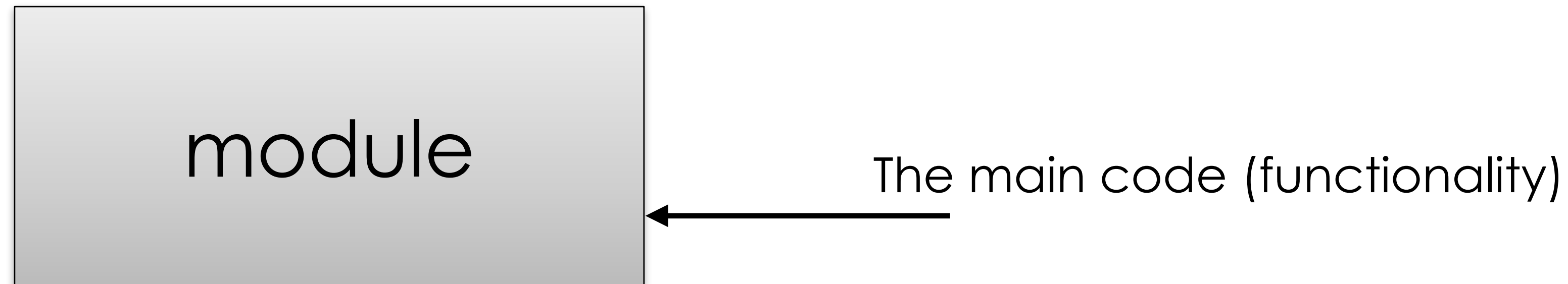
VERILOG BASICS: PORT MODES

- **input**: can only be read
- **output**: can be on either side of assignment

```
module my_module(a,b,c,d);  
    input a,b;  
    output c,d;  
  
    // Module body  
  
endmodule
```

VERILOG BASICS

- Each design in Verilog consists of one main part:



How to describe functionality?

Behavioral

What a module does

Structural

How a module is built

```
module identifier(port_list);
[ports_declaration]

//module_body

endmodule
```

VERILOG BASICS: CONTINUOUS ASSIGNMENT STATEMENTS

assign statement

Describes only combinational logic

These statements are re-evaluated anytime any of the inputs on the right hand side changes

`assign <out> = <in0> <operator> <in1> ... ;`

Example

`assign Y = A + B;`

VERILOG BASICS: CONTINUOUS ASSIGNMENT STATEMENTS

Find the bug in this 2 input AND gate!

```
module assign_example (A,B,Y) ;  
    input A;  
    input B;  
    output Y  
  
    Y = A & B;  
endmodule
```

VERILOG BASICS: CONTINUOUS ASSIGNMENT STATEMENTS

Fixed version

```
module assign_example(A,B,Y);  
    input A;  
    input B;  
    output Y;  
  
    assign Y = A & B;  
endmodule
```

VERILOG CODE EXAMPLE

4x1 multiplexer

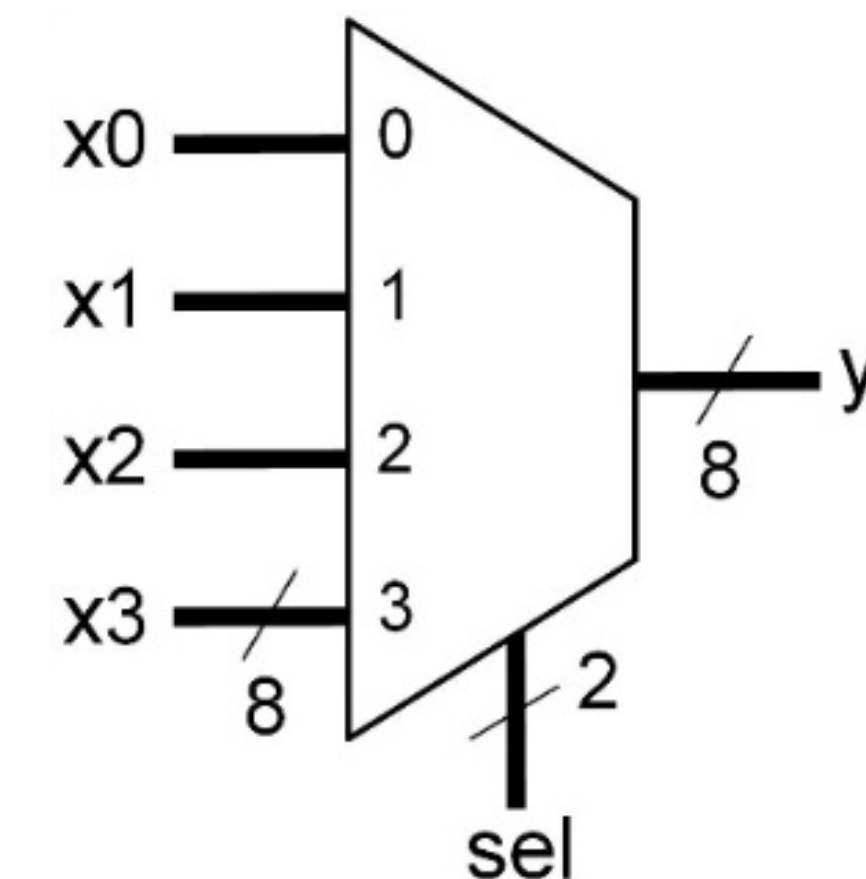
```
module mux_4_1 (x0, x1, x2, x3, sel, y);
  input [7:0] x0,x1,x2,x3;
  input [1:0] sel;
  output [7:0] y;
```

Size (bit-width)

```
  assign y = (sel==2'b00) ? x0:
             (sel==2'b01) ? x1:
             (sel==2'b10) ? x2:
             x3;
```

```
endmodule
```

sel ₁	sel ₀	Y
0	0	X ₀
0	1	X ₁
1	0	X ₂
1	1	X ₃



Break 8 minutes
Return at ...



VERILOG CODE EXAMPLE

```

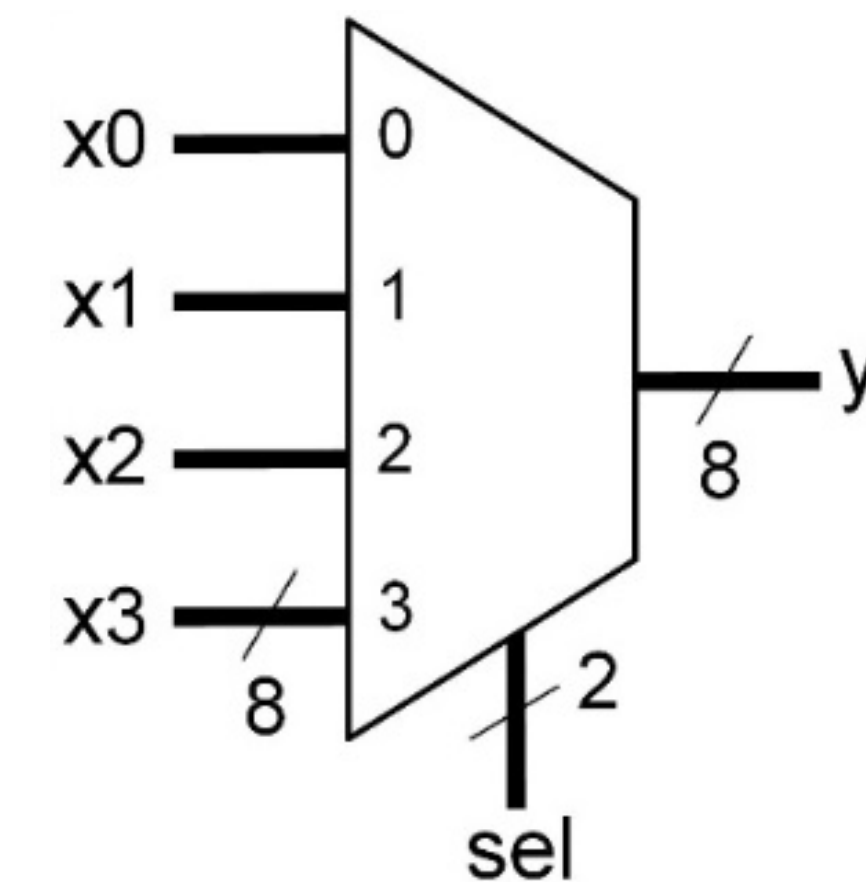
module mux_4_1 (x0,x1,x2,x3,sel,y);
    input [7:0] x0,x1,x2,x3;
    input [1:0] sel;
    output [7:0] reg y;
    always @(x0,x1,x2,x3,sel)
    begin
        case (sel):
            2'b00:
                y = x0;
            2'b01:
                y = x1;
            2'b10:
                y = x2;
            2'b11:
                y = x3;
        endcase
    end
endmodule

```

Sensitivity list

Rule
assign statements
 cannot be used inside
always block

sel ₁	sel ₀	Y
0	0	X ₀
0	1	X ₁
1	0	X ₂
1	1	X ₃



VERILOG CODE EXAMPLE

```

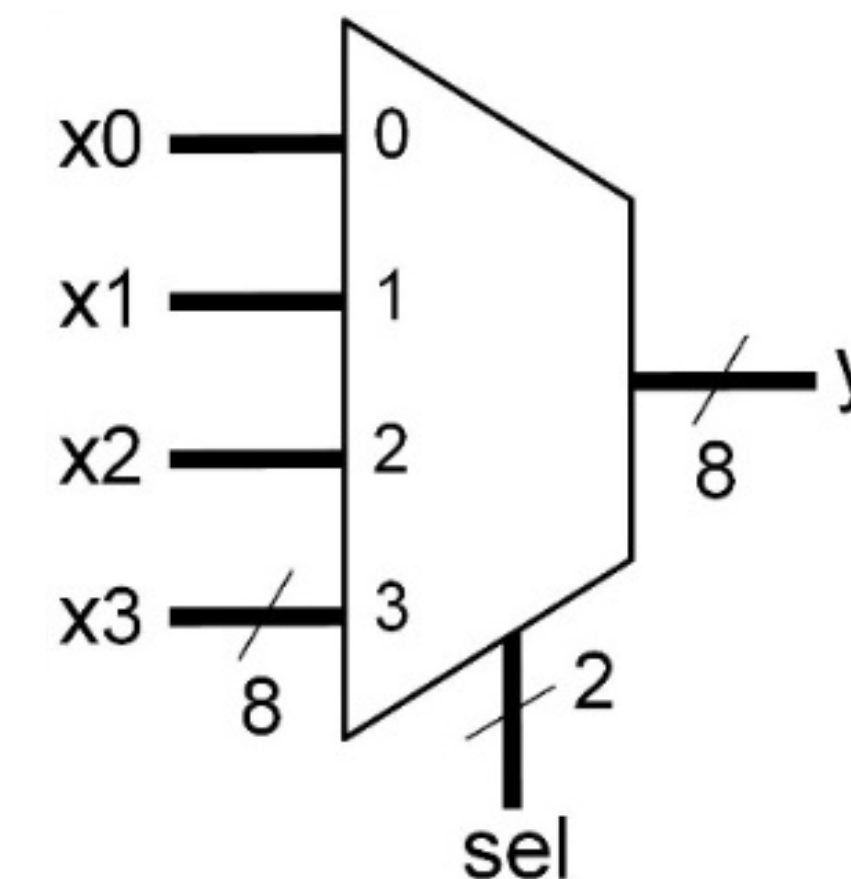
module mux_4_1 (x0,x1,x2,x3,sel,y);
  input [7:0] x0,x1,x2,x3;
  input [1:0] sel;
  output [7:0] reg y;
always @(*)
begin
  case (sel):
    2'b00:
      y = x0;
    2'b01:
      y = x1;
    2'b10:
      y = x2;
    2'b11:
      y = x3;
  endcase
end
endmodule

```

Sensitivity list

Rule
assign statements
 cannot be used inside
always block

sel ₁	sel ₀	Y
0	0	X ₀
0	1	X ₁
1	0	X ₂
1	1	X ₃



SOME VERILOG DATA TYPES

4-state types

•Nets

- Defined by predefined word *wire*, with 4 known values
- Can be driven in concurrent statements
- They change continuously by circuit
- Syntax: Example:

```
wire net_name; //just name
wire S1= 1'b0; //name and initial value
```

•Registers

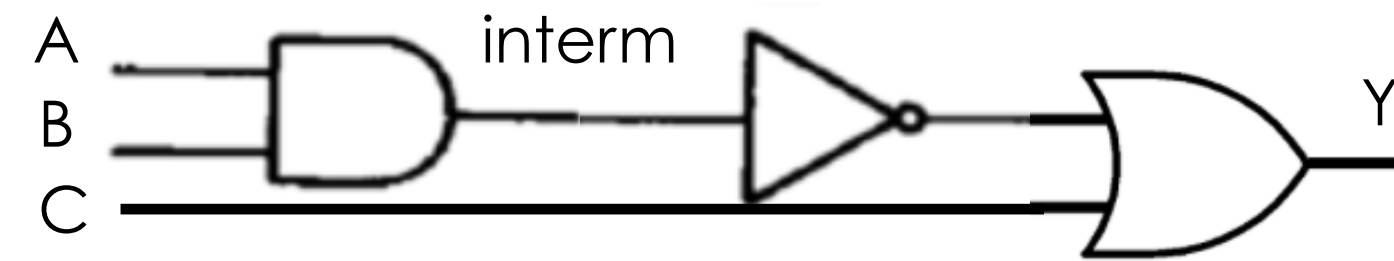
- Defined by predefined word *reg*, with 4 known values
- Can be driven in sequential statements
- In contrast to nets stores values until they are updated
- Syntax: Example:

```
reg reg_name; //just name
reg S1= 1'b0; //name and initial values
```

Value	Definition
0	Logic 0 (false)
1	Logic 1 (true)
X	Unknown
Z	High impedance

VERILOG DATA TYPES

Example



```

module always_ex(A,B,C,Y);
    input A;
    input B;
    input C;
  
```

endmodule

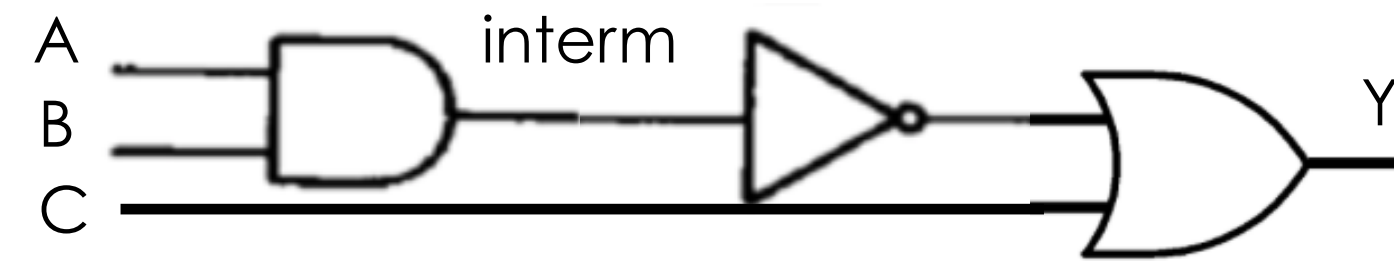
```

module assign_ex(A,B,C,Y);
    input A;
    input B;
    input C;
  
```

endmodule

VERILOG DATA TYPES

Example



```

module always_ex(A,B,C,Y);
  input A;
  input B;
  input C;
  output reg Y;
  
```

endmodule

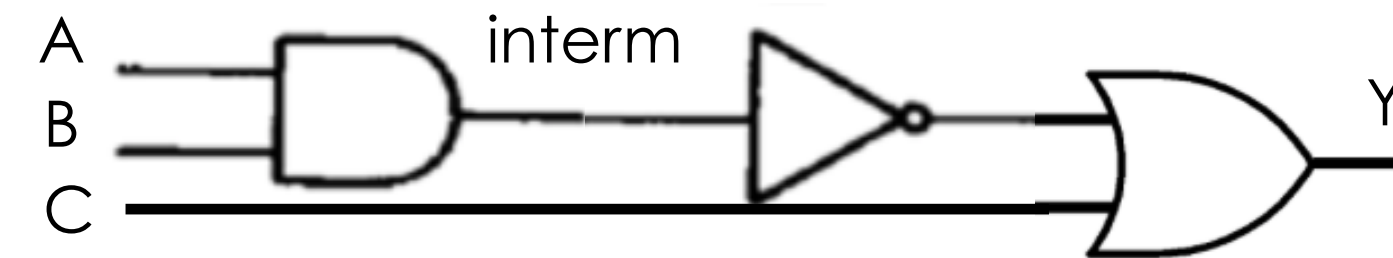
```

module assign_ex(A,B,C,Y);
  input A;
  input B;
  input C;
  output Y;
  
```

endmodule

VERILOG DATA TYPES

Example



```

module always_ex(A,B,C,Y);
  input A;
  input B;
  input C;
  output reg Y;

  reg interm;
  
```

endmodule

```

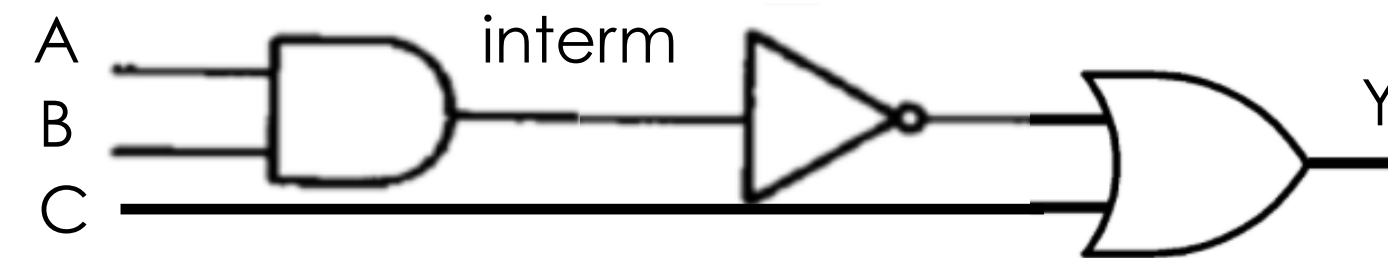
module assign_ex(A,B,C,Y);
  input A;
  input B;
  input C;
  output Y;

  wire interm;
  
```

endmodule

VERILOG DATA TYPES

Example



```

module always_ex(A,B,C,Y);
  input A;
  input B;
  input C;
  output reg Y;

  reg interm;

  always @(*) begin
    interm = A & B;
    Y      = ~interm | C;
  end
endmodule
  
```

Keep its value until
always block is
triggered

```

module assign_ex(A,B,C,Y);
  input A;
  input B;
  input C;
  output Y;

  wire interm;

  assign interm = A & B;
  assign Y      = ~interm | C;

endmodule
  
```

Change continuously in
the circuit

MORE VERILOG DATA TYPES

•Vector

Multiple bits.

A reg or a net can be declared as a vector.

Vectors are declared by brackets []

Examples:

```
reg [7:0] MB1; //8-bit reg vector with MSB=7 LSB=0
wire [0:7] MB2; //8-bit wire vector with MSB=0 LSB=7
reg [3:0] bitslice;
// with initialization
wire [3:0] multiBitWord1 = 4'b1010;
reg [7:0] total = 8'd12; //8 bits and decimal value 12
                        // 8'b0000_1100
```

Referencing vectors

```
a = multiBitWord1[3]; // a = 1
bitslice1 = total[3:0]; // bitslice1 = 4'b1100
bitslice2 = total[7:4]; // bitslice2 = 4'b0000
```

MORE VERILOG DATA TYPES

•Vector

Multiple bits.

A reg or a net can be declared as a vector.

Vectors are declared by brackets []

Examples:

```
reg [7:0] MB1; //8-bit reg vector with MSB=7 LSB=0
wire [0:7] MB2; //8-bit wire vector with MSB=0 LSB=7
reg [3:0] bitslice;
```

•Array

Array as per definition in a collection of elements of same type.

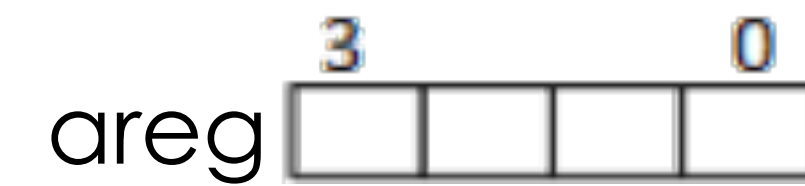
```
reg signed [3:0] anArray [0:4];
//anArray has 5 elements and each element is 4 signed bits
```

Size of each element

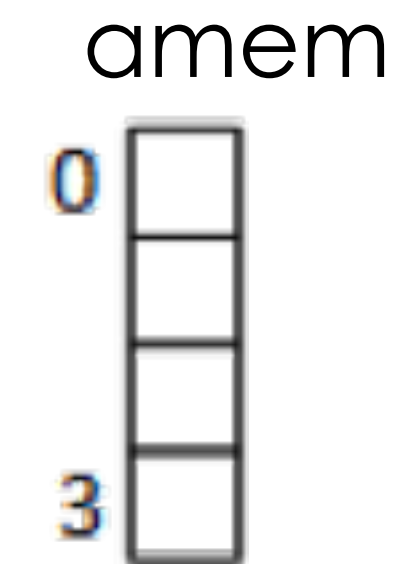
Size of the array

VERILOG DATA TYPES

```
//An 4-bit vector
wire [3:0] areg;
```

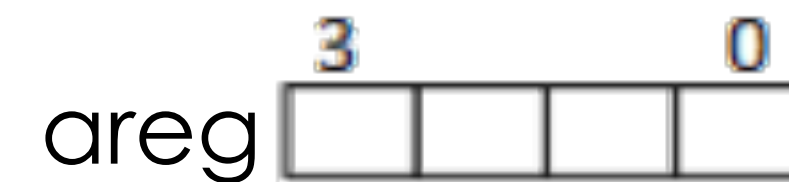


```
//A memory of 4 one-bit elements
reg amem [0:3];
```

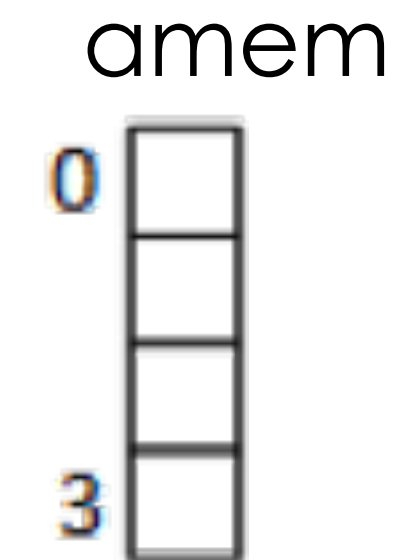


VERILOG DATA TYPES

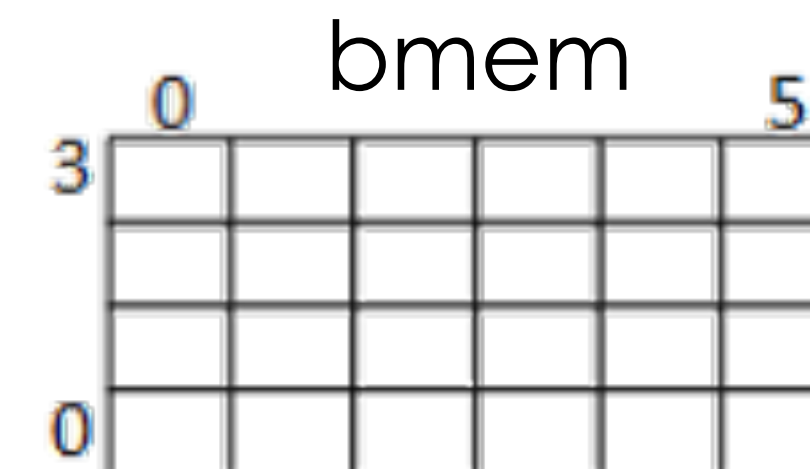
```
//An 4-bit vector
wire [3:0] areg;
```



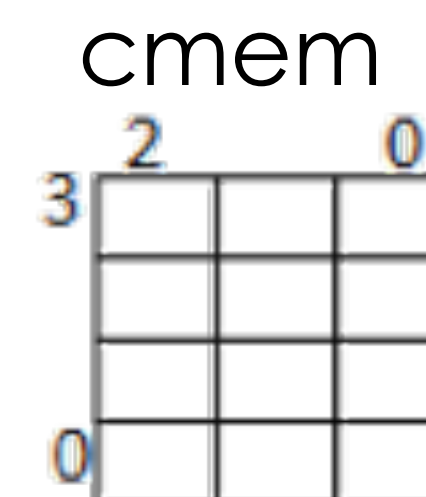
```
//A memory of 4 one-bit elements
reg amem [0:3];
```



```
//A memory of four 6-bit
reg [0:5] bmem [3:0];
```

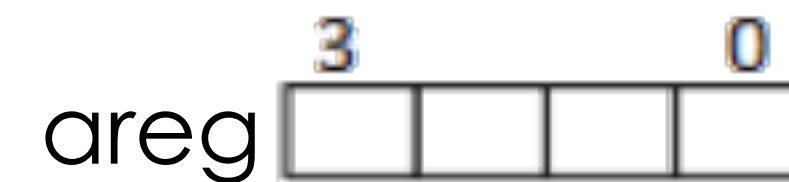


```
//A two dimensional memory of one-bit elements
reg cmem [3:0][2:0];
```

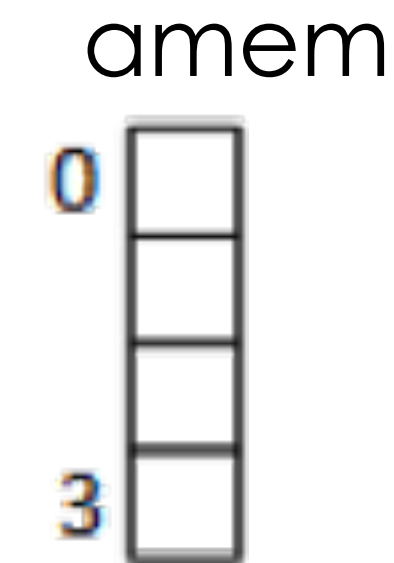


VERILOG DATA TYPES

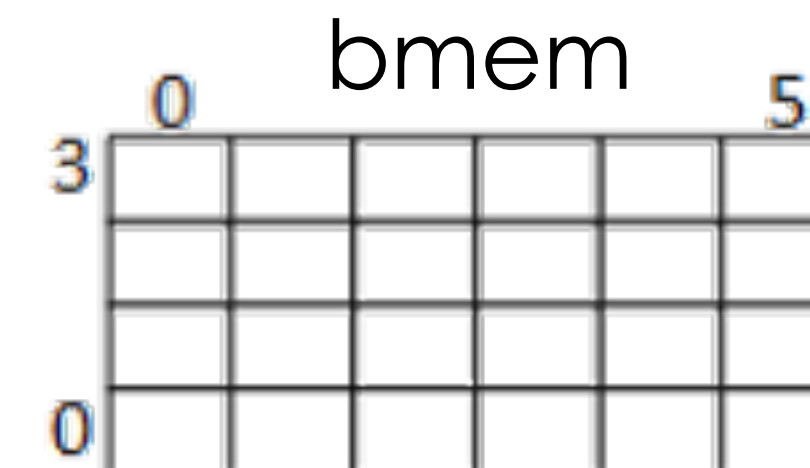
```
//An 4-bit vector  
wire [3:0] areg;
```



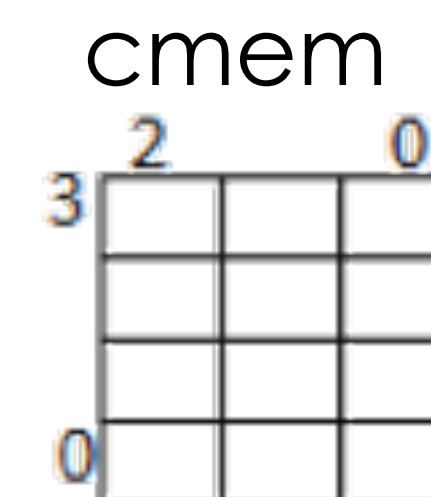
```
//A memory of 4 one-bit elements  
reg amem [0:3];
```



```
//A memory of four 6-bit  
reg [0:5] bmem [3:0];
```



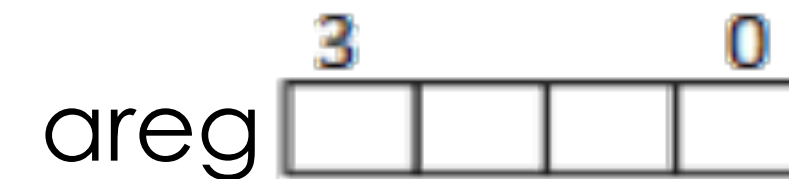
```
//A two dimensional memory of one-bit elements  
reg cmem [3:0] [2:0];
```



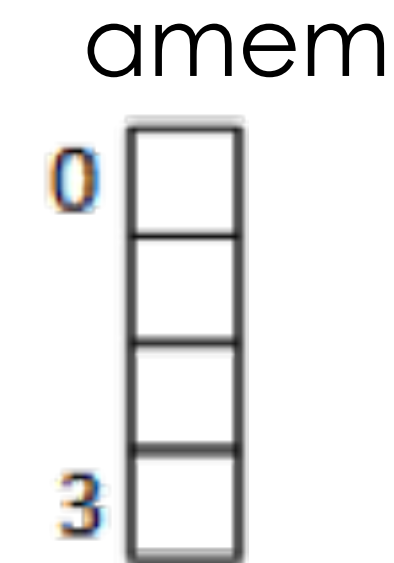
VERILOG DATA TYPES

•Parameter

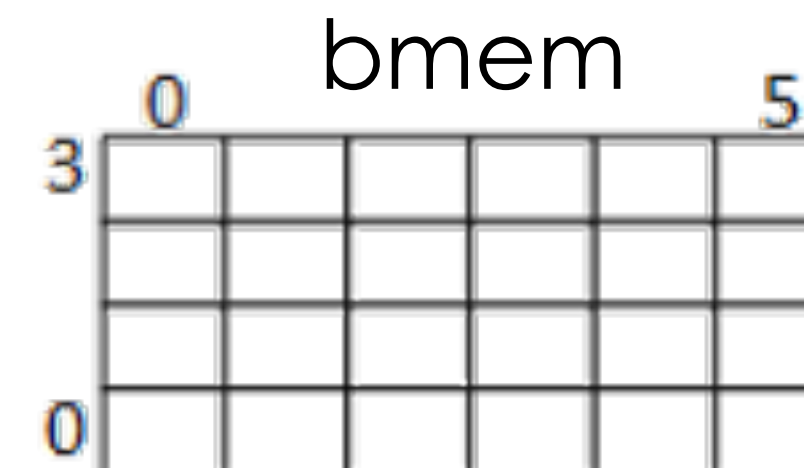
//An 4-bit vector
`wire [3:0] areg;`



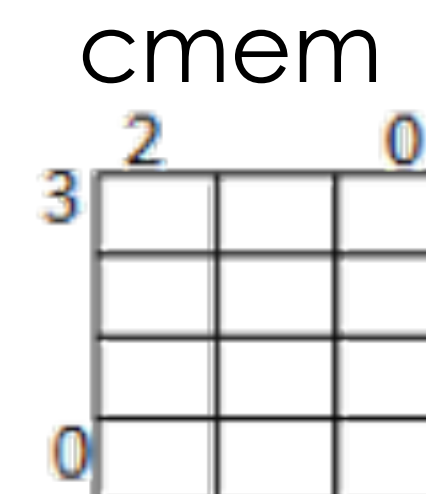
//A memory of 4 one-bit elements
`reg amem [0:3];`



//A memory of four 6-bit
`reg [0:5] bmem [3:0];`



//A two dimensional memory of one-bit elements
`reg cmem [3:0][2:0];`



VERILOG DATA TYPES

•Parameter

Represents a global constant

Declared by a predefined word *parameter*

Example:

```
parameter N = 3;
```

```
//An 4-bit vector
```

```
wire [N:0] areg;
```

```
//A memory of 4 one-bit elements
```

```
reg amem [0:N];
```

```
//A memory of four 6-bit
```

```
reg [0:5] bmem [N:0];
```

```
//A two dimensional memory of one-bit elements
```

```
reg cmem [N:0][2:0];
```